Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University
Alangilan Campus
Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200
Tel Nos.: (+63 43) 425-0139 local 2222 / 2221
E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: http://www.batstate-u.edu.ph

College of Informatics and Computing Sciences

PROJECT PROPOSAL
OOP 101 – OBJECT-ORIENTED PROGRAMMING
Computer Science Department
1st Semester, AY 2026-2027

I. TEAM INFORMATION

Team Name:	ExD		
Team Number:	4	**Section:**	CS 2102

TEAM MEMBERS			
No.	**SR-Code**	**Name**	**GitHub Username**
1	25-07863	Atienza, Ron Xyver	ronxyver
2	25-78190	Manalo, John Efrain V.	chescawithlettertheC
3	25-01536	Manila, Franchesca Denise B.	johnefrainmanalo

II. PROJECT INFORMATION

Project Title
Sumurai

Project Background
Mental arithmetic fluency is foundational to academic success, yet traditional, repetitive drills fail to engage students or build lasting skills. As a result, students calculate slowly, make mistakes, and lose confidence, which can lead to math anxiety and avoidance. Sumurai addresses this by gamifying arithmetic practice, turning routine into an enjoyable, confidence-building activity.

Traditional practice methods rely on passive, feedback-poor drills that hinder calculation speed, lead to errors, and ultimately trigger math anxiety. This issue primarily impacts elementary and junior high school students who are building foundational skills, while also leaving teachers and parents without effective tools to track progress or keep learners motivated. Overcoming this challenge is crucial because basic arithmetic fluency frees up the mental processing capacity needed for complex reasoning in algebra, science, and everyday life. By using timed, game-based practice to eliminate the boredom bottleneck, students can safely build speed and accuracy, replacing anxiety with the confidence needed to tackle higher-level math.

Project Objectives
General Objective
To develop a Java-based application that will help students improve their speed, accuracy, and confidence in mental arithmetic by turning math practice into a simple battle game.

Specific Objectives
The project aims to:

1. Automatically generate arithmetic problems that will test the user's speed and accuracy in mentally solving arithmetic problems.
2. Implement a battle system that displays a battle time countdown, allows six move types, and drives players to victory by defeating their opponent.
3. Provide player statistics that display analysis of their speed and accuracy and battle accomplishments (longest battle victory streak, number of battles won, and shortest time in battle).
4. Apply object-oriented programming fundamentals and principles to the implementation of the application.

Target Users

Target User	How They Will Use the System
Students	Players who will battle against robotic opponents, move by answering math problems, and review their speed and accuracy.
Educators and Parents	Use Sumurai as an engaging practice tool for themselves and for their students, and to oversee their students' progress.

Proposed Features

Feature	Description
Battle System	Fighters start with 100% HP (health points). Players can choose any move, then solve a generated problem to perform it. All moves are subject to cooldown. **Cooldown is the interval of a move in seconds.* **Basic Attack**: Deals a certain amount of damage. **Special Attack**: Deals a certain amount of damage. **Buff**: Increases the damage of attacks by a certain percentage and decreases the cooldown of moves by a certain percentage. **Debuff**: Lowers the opponent's defense by a certain percentage and increases the opponent's cooldown of moves. **Shield**: Blocks a certain amount of damage. **Heal**: Restores a certain amount of HP.
Math Problem Generator	For moves to be performed, players shall answer a generated math problem correctly. Problems involve: numerical comparison, addition, subtraction, multiplication, division, algebraic formulation, simplification of fractions, and conversion of fractions to decimals and vice versa. Players shall type in correct answers to successfully perform moves.

Robotic Opponent	Opponents that act on their own whose moves are picked randomly and are subject to interval and success rate.
Battle Time Countdown	Display a timer for the battle that will put players under pressure and thrill to defeat opponents.
Results and Progress Summary	After each battle, show victory or defeat (and whether it ended by knockout or time-out), accuracy (%), average response time, and longest streak. Session statistics are kept in memory; saving the best results to a text file is a stretch goal.
Difficulty Level	Players select difficulty by selecting levels. No players can advance if the current level is not completed. Each level corresponds to a harder level of robotic opponents.
Graphical User Interface	Windows login page, home page, battle page, battle results page, dashboard page, and leaderboard page.

III. JAVA AND OOP CONCEPTS

OOP Concept	How It Will Be Used
Classes and Objects	Each game element is a class: Player, Opponent, Move types, MathProblem, ProblemGenerator, BattleEngine, and BattleStats. Objects are created at runtime, e.g., a new MathProblem each time the player chooses a move and one Opponent per battle.
Methods	Behavior is placed in methods such as generateProblem(), checkAnswer(), takeDamage(), heal(), execute(), isDefeated(), tick(), updateCooldown(), and getAccuracy(). Overloaded generateProblem() versions may take a difficulty and/or an operation.
Encapsulation	Attributes such as HP, shield, buff/debuff timers, and score are private and accessed through getters and controlled methods. For example, takeDamage() and heal() keep HP between 0 and max HP so no other class can set invalid values.
Inheritance	Player and Opponent extend the abstract GameCharacter (shared HP, shield, status effects). BasicAttack, SpecialAttack, Buff, Debuff, Shield, and Heal extend the abstract Move (shared name, cooldown, and problem difficulty). Opponent types (Apprentice, Ronin, Shogun) extend Opponent.

Polymorphism	BattleEngine calls move.execute(user, target) on any Move without knowing its type; each subclass overrides it with its own effect (damage, heal, shield, etc.). Opponent subclasses override chooseMove() to apply different AI strategies.
Abstraction	Move and GameCharacter are abstract classes that define what every move/character must do (execute(), takeDamage()) while hiding how. The GUI only talks to BattleEngine and does not need to know the battle rules or answer-checking details.
Exception Handling	try-catch handles NumberFormatException when the answer field contains letters or is empty, and avoids division by zero in the generator. A custom InvalidAnswerException may be used for bad input. Errors show a friendly message instead of crashing the game.
Arrays/Data Handling	An ArrayList<Move> stores each character's moves, an array stores the three opponents and difficulty settings, and an ArrayList<Double> stores response times to compute averages. A battle log list feeds the GUI text area.

IV. PROPOSED CLASSES

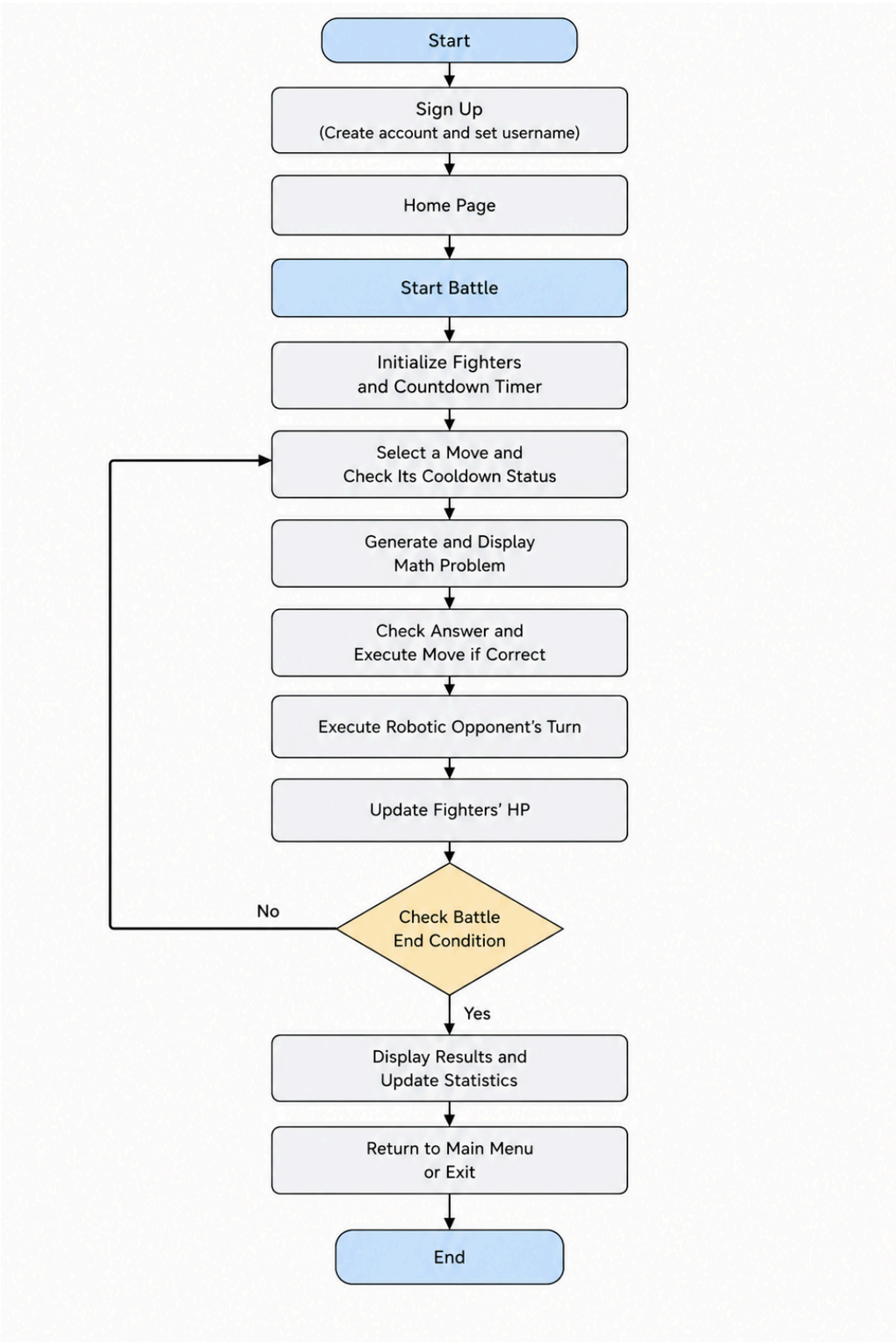
Class	**Purpose**	**Main Attributes**	**Main Methods**
Class	Purpose	Main Attributes	Main Methods
GameCharacter (abstract)	Parent of all fighters; holds shared battle state.	name, hp, maxHp, shield, damageBonus, cooldownModifier, moves (ArrayList)	takeDamage(), heal(), addShield(), applyBuff(), applyDebuff(), updateEffects(), isDefeated()
Player	The human student's character.	totalCorrect, totalAnswered, streak	recordAnswer(), getAccuracy()
Opponent (abstract) and Apprentice, Ronin, Shogun	Computer-controlled fighters with different strengths.	accuracyRate, actionInterval, timeSinceLastAction	chooseMove(), shouldAct(), simulateAnswer()
Move (abstract)	Parent of all moves.	name, cooldown (seconds), currentCooldown, power, problemLevel	execute(user, target), isReady(), startCooldown(), updateCooldown()

BasicAttack, SpecialAttack, Buff, Debuff, Shield, Heal	Concrete moves with their own effects.	damage / healAmount / duration (varies per move)	execute() (overridden)
MathProblem	Represents one problem and its answer.	question, correctAnswer, operation, timeLimit	checkAnswer(int), getQuestion()
ProblemGenerator	Creates problems by difficulty and operation.	random, difficulty	generateProblem(), generateProblem(level)
Difficulty (enum)	Stores range and time limit per level.	EASY, NORMAL, HARD, maxNumber, timeLimit	getTimeLimit(), getMaxNumber()
BattleEngine	Runs the battle clock, applies moves, and decides the winner (knockout or time-out).	player, opponent, timeRemaining, activeProblem, battleLog, stats	startBattle(), tick(), chooseMove(), submitAnswer(), checkWinner(), getTimeoutWinner()
BattleStats	Tracks results for the summary screen.	responseTimes (ArrayList), correct, wrong, longestStreak	addResult(), getAverageTime(), getAccuracy()
GameWindow, MenuPanel, BattlePanel, ResultPanel (Swing)	GUI screens and event handling.	buttons, hpBars, timerLabel, answerField, logArea	showMenu(), updateHpBars(), startTimer(), onSubmit()
Main	Entry point that launches the game window.	–	main()

The classes may be changed as the project is developed.

V. PROGRAM FLOW

Users log in using their credentials, then land on the home page where they will select play, statistics, or exit. Next, users can select a level to fight robotic opponents and practice their mental math problem-solving capabilities in terms of speed and accuracy. Afterwards, users can review their statistics and see how well they played in battle. Player statistics are updated to display their progress.



VI. TEAM RESPONSIBILITIES

Member	Assigned Tasks
Ron Xyver Atienza	In charge of general supervision of all tasks and implementation of the project. Leads and oversees the application of Java fundamentals and OOP concepts.
John Efrain V. Manalo	Responsible for developing the system requirements, designs, diagrams, and overall structure of the application.
Franchesca Denise B. Manila	Handles the preparation of the project proposal and technical documentation, including program flow, system overview, diagrams, and README.

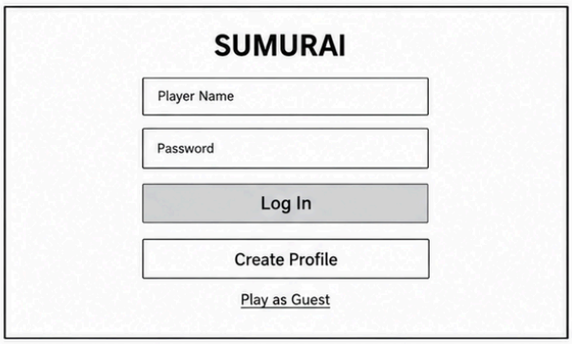
VII. DEVELOPMENT SCHEDULE

Week	Activity	Expected Output
Week 1	Planning and requirements	Project proposal
Week 2	System and class design	Class/UML diagram
Week 3	Initial coding	Basic classes
Week 4–5	Feature development	Working features
Week 6	OOP implementation	Completed OOP features
Week 7	Testing and debugging	Tested application
Week 8	Documentation and presentation	Final project

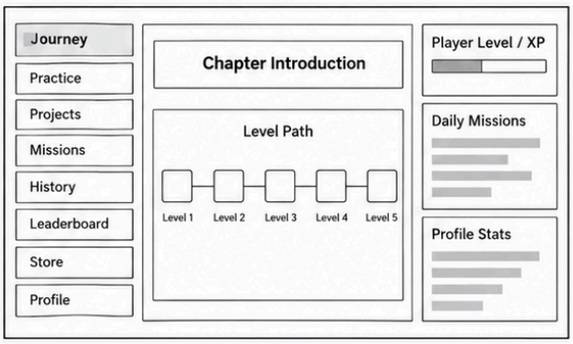
The schedule may be adjusted based on the course calendar.

VIII. EXPECTED OUTPUT

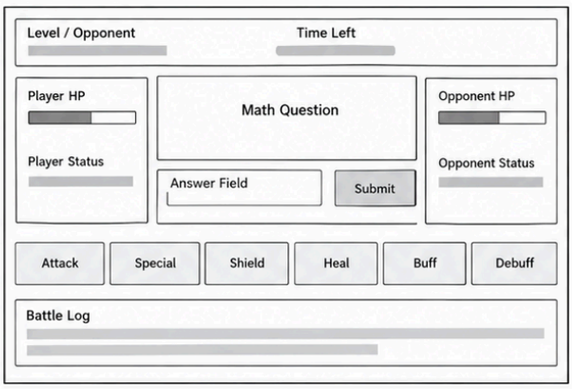
1. Login / Sign Up Page



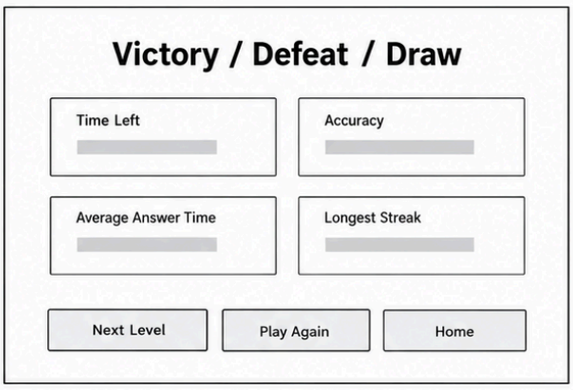
2. Home Page / Journey



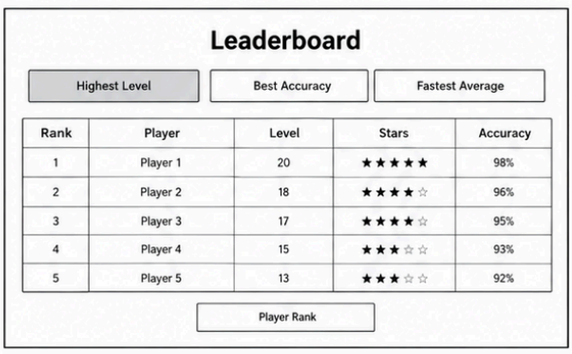
3. Battle Page



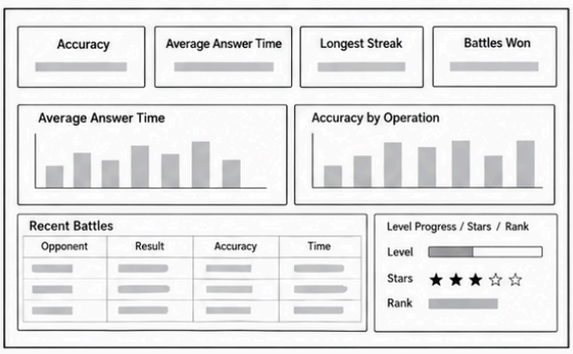
4. Battle Results Page



5. Leaderboard Page



6. Dashboard / Progress Page



The team expects to produce a functional Java application that shall help users to improve their speed, accuracy, and confidence with regard to mental arithmetic problem-solving by eliminating repetitive traditional practice and turning it into an enjoyable gaming experience. The application will demonstrate the appropriate use of Java and object-oriented programming concepts.

IX. PROJECT LIMITATIONS

The project does not cover the following:

1. Player-vs-player battles
2. Topics beyond the included arithmetic problems

3. Mobile or web version
4. Fast-action arcade controls such as movement, jumping, or dodging; the only action is answering math problems, and the player answers one problem at a time.
5. Advanced graphics, animation, and sound; the GUI uses simple Swing components and basic images.

X. GITHUB REPOSITORY

Repository Name: Sumurai
Link to Repository: https://github.com/ronxyver/Sumurai.git

XI. APPROVAL

Prepared by:

Name	Signature
Ron Xyver Atienza	
John Efrain V. Manalo	
Franchesca Denise B. Manila	

Instructor's Comments:

_____

_____

_____

_____

_____

_____

_____

_____

Project Status:

☐ Approved
☐ Approved with Revisions
☐ Revise and Resubmit

Name of Instructor:

Ms. FATIMA MARIE P. AGDON, MSCS

Date Signed: _____